ComplaintIQ: AI-Powered Customer Complaint Classification & Resolution

1. Problem understanding

Support teams receive large volumes of free-text complaints. Done by hand, each one has to be read, labelled, prioritised and routed, and a reply has to be written. This is slow and inconsistent. Three costs follow:

- Urgent cases (fraud, safety, legal threats) wait in the same queue as routine ones.
- The same customer complains several times, so agents duplicate work.
- A systemic problem, such as a courier failing in one city or a faulty product batch, is noticed only after hundreds of complaints.

The goal: a platform that understands each complaint, classifies it, scores sentiment, urgency and severity, extracts key facts, routes it, drafts a grounded reply for staff approval, groups duplicates, flags emerging issues, and learns from agent corrections.

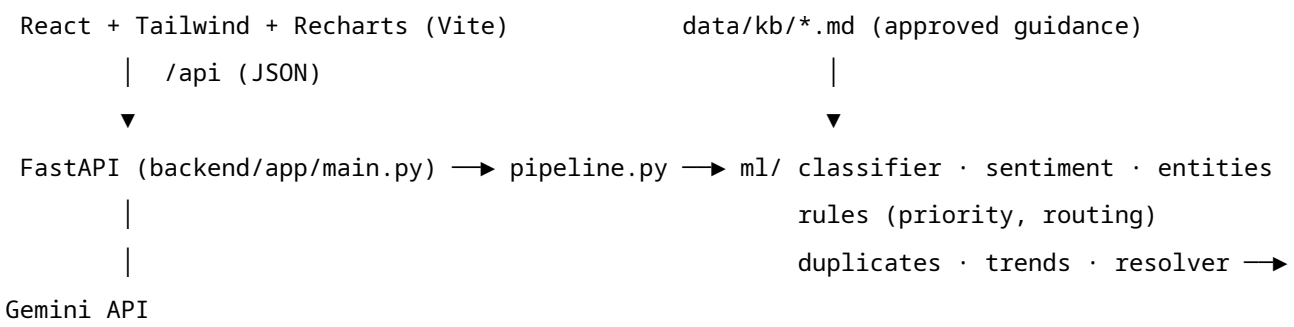
2. Proposed solution

Every complaint goes through one pipeline (`backend/app/pipeline.py`). Each step produces a result **and the reason for it**:

Step	Method	Explanation shown to staff
Intake	Form or CSV, HTML/URL stripping, length and duplicate-row checks	Per-row error report for CSV
Category (6 classes)	`all-MiniLM-L6-v2` sentence embeddings + logistic regression	Confidence for every class + word-occlusion importance (highlighted words)
Sentiment (4 levels)	`cardiffnlp/twitter-roberta-base-sentiment-latest`, escalated to *highly negative* by intensity markers	Class probabilities
Emotion / frustration	`j-hartmann/emotion-english-distilroberta-base`	7 emotion scores; frustration = anger + disgust + ½ sadness

Step	Method	Explanation shown to staff
Entities	Regex (order ID, amount, email, phone) + catalogue gazetteer (product, city) + spaCy NER (dates, other places, money)	Entity chips
Priority, urgency, severity	Additive business-rule score (fraud, safety, legal threat, urgency, amount, sentiment, frustration, repeat complaint); keywords preceded by "not/no/never" are ignored. The same factors split into a 0-1 urgency score (how soon) and severity score (how bad)	Every point listed with its reason; urgency and severity shown as rule scores, not probabilities
Routing	Category → department, with overrides for fraud and product safety	Routing reason text
Duplicates	Cosine similarity of embeddings + identity rules (customer / order ID)	Similarity %, link to the original, human confirm/reject
Similar complaints	Top-5 semantic neighbours across all customers	Similarity %
Resolution	Retrieve the 2 closest approved playbooks (`data/kb`, +0.15 similarity bonus for the predicted category), Gemini drafts a reply grounded only in them; the complaint is passed as delimited data; cited playbooks not in the retrieved set are dropped; template fallback when the key, call or reply shape fails	Cited playbooks; staff must approve, edit or reject (sending is out of scope)
Trends	Last 72 h vs previous 14 days per category, product and location (windows end at the newest complaint, never after now; needs 17 days of history); z-score ≥ 2.5 and ≥ 6 complaints	Alert text with rates, z-score and % increase
Feedback	Every confirmation or correction is logged (the latest verdict per complaint and field counts); per-field agreement, weak spots for any field and predicted value, retrain with guard	Evaluation dashboard

3. Architecture



```
PostgreSQL (SQLAlchemy; SQLite fallback)
  complaints: text, predictions, scores, explanations, entities, embedding, draft, status
  feedback:   complaint_id, field, predicted, corrected, comment
```

Offline scripts in `ml/`:

- `generate_data.py` builds the dataset and the demo stream.
- `train.py` trains and compares the classifiers.
- `evaluate.py` evaluates the other components.
- `seed_db.py` loads the demo stream through the live pipeline.

3a. Deployment

A multi-stage `Dockerfile` builds the Vite frontend, installs the CPU PyTorch stack with `uv`, bakes the three pretrained models into the image and starts `uvicorn` on port 7860. FastAPI serves `frontend/dist` (landing page at `/`, console at `/app`) from the same origin as `/api`, so no CORS or proxy is needed. The database is empty at start; `DATABASE_URL` selects PostgreSQL, otherwise SQLite.

4. Technology choices

- **Sentence Transformers (MiniLM) + scikit-learn LR.** One 384-d embedding serves classification, duplicate detection, similar-complaint search and RAG retrieval. It is fast on CPU, beat the TF-IDF baseline by 11 accuracy points, and can be retrained in seconds from cached embeddings. Fine-tuning a transformer would cost far more compute for little gain at this data size.
- **Pretrained transformers (PyTorch) for sentiment and emotion.** No labelled sentiment data exists for this domain, and both are mature public models.
- **Rules for priority and routing.** These are business decisions that must be auditable and editable. The ML signals (sentiment, frustration, entities, duplicates) feed into the rules.
- **spaCy + regex for entities.** IDs and amounts have fixed formats, where regex is exact. spaCy covers free-form dates and places.
- **RAG + Gemini for drafting.** Drafts are grounded in approved guidance, cite their sources, and are never sent without approval. The free-tier API key carries no billing risk. Without a key, a deterministic template keeps the feature working.
- **PostgreSQL via SQLAlchemy.** Relational data with JSON columns for model outputs. `DATABASE_URL` switches to SQLite for zero-setup runs.

5. Implementation notes

- **Explainability.** Category explanations use occlusion. Each word is removed in turn, the text is re-scored, and the drop in predicted-class probability is that word's importance. This is faithful to the real embedding model, not a proxy model. Only the first 80 words are scored, to bound latency.
- **Duplicate definition.** A duplicate is *the same case raised again*: similarity ≥ 0.80 with a shared customer or order ID, or ≥ 0.88 when identity is unknown. Different customers or different order IDs

are never duplicates, even with near-identical wording. That case is a *trend*, and it appears under "Similar complaints". This rule came from evaluation: in an early run (not saved as an artifact), plain cosine similarity flagged hundreds of different-customer complaints as duplicates, at a precision near 0.2. With the identity rules, precision is 0.93 at recall 1.00 (`evaluation.json`).

- **Human in the loop.** Category, sentiment, priority and department can each be corrected. Drafts need approval. Duplicate matches need confirmation. The original model output is kept in `analysis.original`, so agreement is measured against the model, not the last edit.
- **Safe retraining.** `/api/model/retrain` refits on training data plus corrected complaints (weight ×3). It deploys only if held-out test accuracy is within 1 point of the **original** model, so repeated retrains cannot drift down a point at a time. The retrained model is written to `classifier_retrained.joblib` (git-ignored); deleting it restores the original.

6. Results

Full numbers: `backend/artifacts/classifier_metrics.json` and `evaluation.json`. They are also shown on the in-app **Evaluation** page.

Category classification (test set n = 1,537; synthetic test templates never seen in training)

Model	Accuracy	Macro-F1
TF-IDF + LR (baseline)	81.3%	0.804
MiniLM + LR (deployed)	**92.7%**	**0.923**

Per-class F1 (MiniLM): payment 1.00, account 0.97, technical 0.92, refund 0.90, delivery 0.89, product 0.85.
By source: Bitext 99.8%, unseen synthetic templates 89.7%.

Other components

Component	Metric	Result
Sentiment (4-level)	accuracy vs generation tone, n = 500	65.4%
Sentiment (3-level, highly negative folded into negative)	accuracy	67.4%
Entities: order ID / product / amount / location	F1	1.00 / 1.00 / 1.00 / 1.00
Duplicates (55 injected re-submissions)	precision / recall / F1	0.93 / 1.00 / 0.96 (threshold sweep 0.5-0.9: F1 0.91-0.97)

Component	Metric	Result
Trend spikes (Chennai delivery, SmartWatch X2 defect + their categories)	recall / false alarms	4/4 found / 0

7. Limitations

- **Synthetic data.** Synthetic complaints come from 72 templates. Results on them, especially entity F1 = 1.0, overstate real-world performance. Bitext adds real phrasing but is mostly requests rather than angry complaints, and it has no "product" class.
- **Sentiment labels are weak.** The tone of the template wrapper is not true sentiment. A politely worded complaint about a broken product is still negative, and the model says so. Most "errors" are neutral→negative, which is arguably correct.
- **Priority is rule-based.** No ground truth for priority exists. The weights are hand-set, and they are explained rather than learned.
- **Duplicate threshold** was chosen on the same demo stream it is evaluated on (no separate tuning split).
- **Sentiment score is partly circular.** The *highly negative* escalation looks for words (worst, unacceptable, lawyer, never shop…) that the synthetic generator also uses to write highly negative complaints, so 65.4% is optimistic.
- **Mild Bitext leak.** 53 of 457 Bitext test rows have token-Jaccard ≥ 0.8 with a training row, which partly explains Bitext's 99.8%. The dataset revision is not pinned.
- **Retrain gate** reuses the offline test set, and the seeded demo complaints are built from test templates, so corrections on them leak into the gate.
- **Keyword rules** use a 3-word negation window, not a parser.
- **No authentication.** Anyone who can reach the deployment can read and submit complaints. Guards in place: host allow-list (`ALLOWED_HOSTS`), writes refused from foreign browser origins, console not frameable. Put it behind SSO before real use. The hosted instance starts with an empty database; the free tier's disk is ephemeral, so data resets on restart.
- **English only, CPU only.** Seeding about 1,200 complaints takes about 3-5 minutes on CPU.
- **LLM output** is constrained by prompt and retrieval but not formally verified. Staff approval is the safeguard.

8. Future improvements

- Collect real labelled complaints and fine-tune the encoder on them. Learn priority weights from agent corrections.
- Use pgvector for approximate-nearest-neighbour duplicate search at scale.
- Add multilingual support (Tamil and Hindi) with a multilingual encoder.
- Use active learning: send low-confidence predictions to reviewers first.
- Add real sending through email or WhatsApp after approval, plus SLA timers per priority.
- Add seasonality-aware trend detection (weekday effects, sales events) instead of a flat 14-day baseline.